

程序设计实践

The Practice of Programming

接口(Interfaces)

接口 (界面)

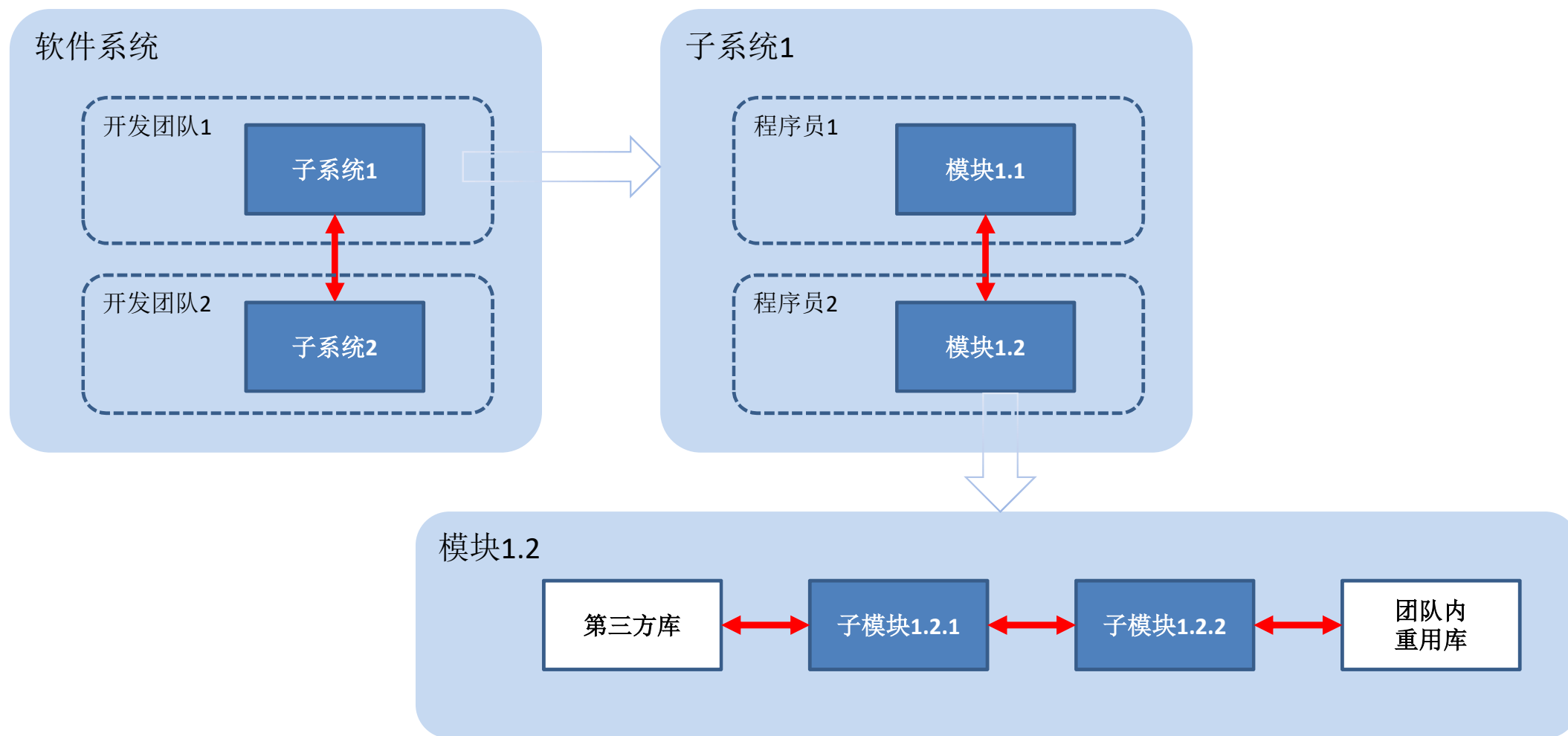
程序与程序之间的接口

库函数，共享数据，通信协议等

人与程序之间的接口

操作命令，配置文件，错误信息，日志
图形用户界面(GUI)
语音，视频

程序与程序之间的接口



库函数接口 – 公共库函数接口

hello_world.c

```
#include <stdio.h>

void main()
{
    printf("Hello World!\n");
}
```

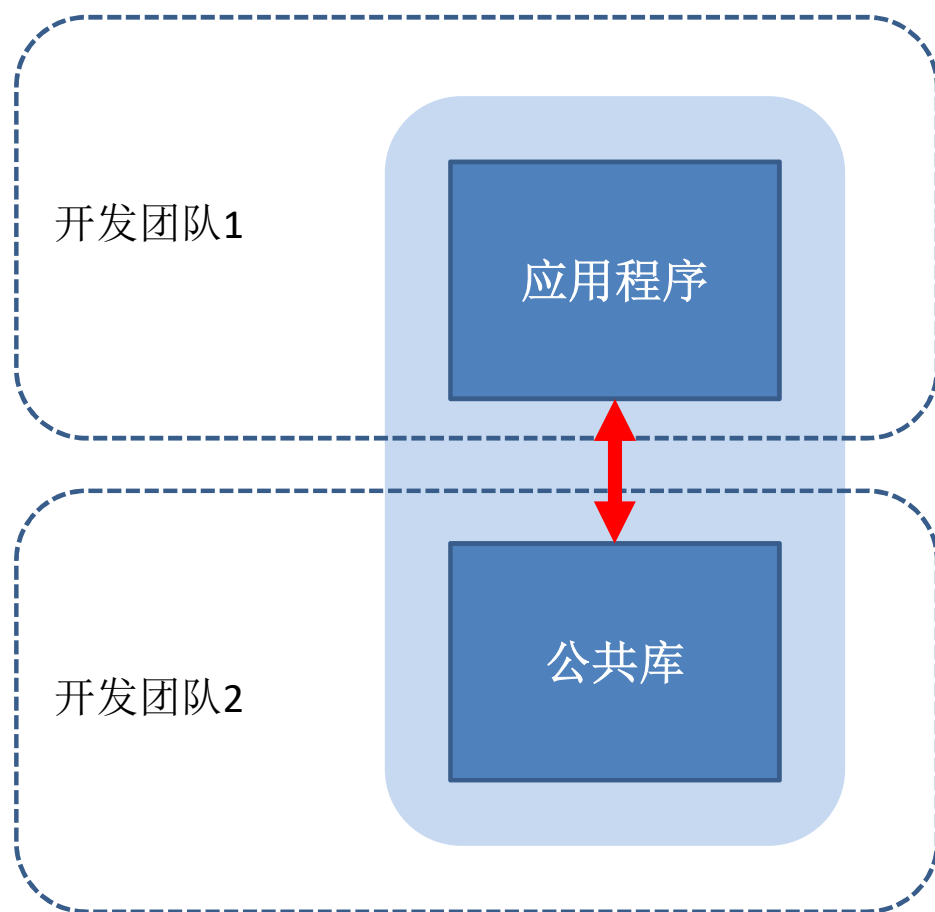
/usr/include/stdio.h (/usr/lib64/libc.a)

```
...
/* Write formatted output to stdout.
   This function is a possible cancellation point and therefore not
   marked with __THROW.  */
extern int printf (const char *__restrict __format, ...);
...
```

接口说明:

- 头文件
- 联机man
- 接口说明文档
- 编程指南

库函数接口 – 组织内的库函数接口



接口说明:

- 头文件
- 接口说明文档
- 编程指南

库函数接口 - 范例

```
QSORT(3)                                Linux Programmer's Manual    QSORT(3)

NAME
    qsort - sorts an array

SYNOPSIS
    #include <stdlib.h>

    void qsort(void *base, size_t nmem, size_t size,
               int(*compar)(const void *, const void *));

DESCRIPTION
    The qsort() function sorts an array with nmem elements of size size. The base argument points to the start of the array.

    The contents of the array are sorted in ascending order according to a comparison function pointed to by compar, which is called with two arguments that point to the objects being compared.

    The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second. If two members compare as equal, their order in the sorted array is undefined.

RETURN VALUE
    The qsort() function returns no value.

CONFORMING TO
    SVr4, 4.3BSD, C89, C99.

NOTES
    Library routines suitable for use as the compar argument include alphasort(3) and versionsort(3). To compare C strings, the comparison function can call strcmp(3), as shown in the example below.

EXAMPLE
    For one example of use, see the example under bsearch(3).

    Another example is the following program, which sorts the strings given in its command-line arguments:

    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    #include <assert.h>

    static int
    cmpstringp(const void *p1, const void *p2)
    {
        /* The actual arguments to this function are "pointers to
         * pointers to char", but strcmp(3) arguments are "pointers
         * to char", hence the following cast plus dereference */

        return strcmp(*(char * const *) p1, *(char * const *) p2);
    }
```

C标准库中提供的快速排序库
函数接口所提供的联机帮助

} 函数原型

} 函数功能简要说明

} 函数返回值说明

} 示例

库函数接口 - 范例

QDialog Class Reference

The QDialog class is the base class of dialog windows. [More...](#)

```
#include <qdialog.h>
```

Inherits [QWidget](#).

Inherited by [QColorDialog](#), [QErrorMessage](#), [QFileDialog](#), [QFontDialog](#), [QInputDialog](#), [QMessageBox](#), [QMotifDialog](#), [QProgressDialog](#), [QTabDialog](#), and [QWizard](#).

[List of all member functions.](#)

Public Members

- explicit [QDialog](#) ([QWidget](#) * parent = 0, const char * name = 0, bool modal = FALSE, WFlags f = 0)
- [~QDialog](#) ()
- enum [DialogCode](#) { Rejected, Accepted }
- int [result](#) () const
- virtual void [show](#) ()
- void [setOrientation](#) (Orientation orientation)
- Orientation [orientation](#) () const
- void [setExtension](#) ([QWidget](#) * extension)
- [QWidget](#) * [extension](#) () const
- void [setSizeGripEnabled](#) (bool)
- bool [isSizeGripEnabled](#) () const
- void [setModal](#) (bool modal)
- bool [isModal](#) () const

Public Slots

QT的帮助文档中，有关QDialog类的参考手册（部分截图），包含：

- **include**文件
- 继承自哪个类
- 被哪些类继承
- **public**成员函数
- 公共插槽
- 详细说明
- ...

等信息

库函数接口 - 范例

ØMQ/4.3.1 API Reference

v4.3 master | v4.3 stable | v4.2 stable | v4.1 stable | v4.0 stable | v3.2 legacy

- `zmq` - ØMQ lightweight messaging kernel
- `zmq_atomic_counter_dec` - decrement an atomic counter
- `zmq_atomic_counter_destroy` - destroy an atomic counter
- `zmq_atomic_counter_inc` - increment an atomic counter
- `zmq_atomic_counter_new` - create a new atomic counter
- `zmq_atomic_counter_set` - set atomic counter to new value
- `zmq_atomic_counter_value` - return value of atomic counter
- `zmq_bind` - accept incoming connections on a socket
- `zmq_close` - close ØMQ socket
- `zmq_connect` - create outgoing connection from socket
- `zmq_ctx_destroy` - terminate a ØMQ context
- `zmq_ctx_get` - get context options
- `zmq_ctx_new` - create new ØMQ context
- `zmq_ctx_set` - set context options
- `zmq_ctx_shutdown` - shutdown a ØMQ context
- `zmq_ctx_term` - terminate a ØMQ context
- `zmq_curve_keypair` - generate a new CURVE keypair
- `zmq_curve_public` - derive the public key from a private key
- `zmq_curve` - secure authentication and confidentiality
- `zmq_disconnect` - Disconnect a socket
- `zmq_errno` - retrieve value of errno for the calling thread
- `zmq_getsockopt` - get ØMQ socket options
- `zmq_gssapi` - secure authentication and confidentiality
- `zmq_has` - check a ZMQ capability

ZeroMQ的应用程序接口（API）参考手册

库函数接口 - 范例

网络SDK开发手册

隐藏

上一步

前进

主页

打印

选项(O)

目录(C)

索引(N)

搜索(S)

收藏夹(I)

前言

内容简介

设备支持的接口概况

接口定义

- SDK初始化
 - CLIENT_Init
 - CLIENT_Cleanup
 - CLIENT_GetSDKVersion
 - CLIENT_GetLastError
 - CLIENT_SetAutoReconnect
 - CLIENT_SetConnectTime
 - CLIENT_SetNetworkParam
- 登陆设备
 - CLIENT_LoginWithHighLevelSecurity
 - CLIENT_Logout
- 实时监视
- 数据保存
- 视频抓图
 - CLIENT_CapturePictureEx
 - CLIENT_SetSnapRevCallBack
 - CLIENT_SnapPictureEx
- 回放和下载
- 回放控制
- 语音对讲
- 云台控制

网络SDK开发手册

CLIENT_LoginWithHighLevelSecurity

登录设备

```
LLONG CLIENT_LoginWithHighLevelSecurity(  
    NET_IN_LOGIN_WITH_HIGHLEVEL_SECURITY *pstInParam,  
    NET_OUT_LOGIN_WITH_HIGHLEVEL_SECURITY *pstOutParam,  
);
```

Parameters

[in] pstInParam
设备登录输入参数

[out] pstOutParam
设备登录输出参数

Return Values

成功返回非0(登录句柄), 失败返回0。

Remarks

一般在SDK初始化后调用, 与设备进行各种业务功能的前提。

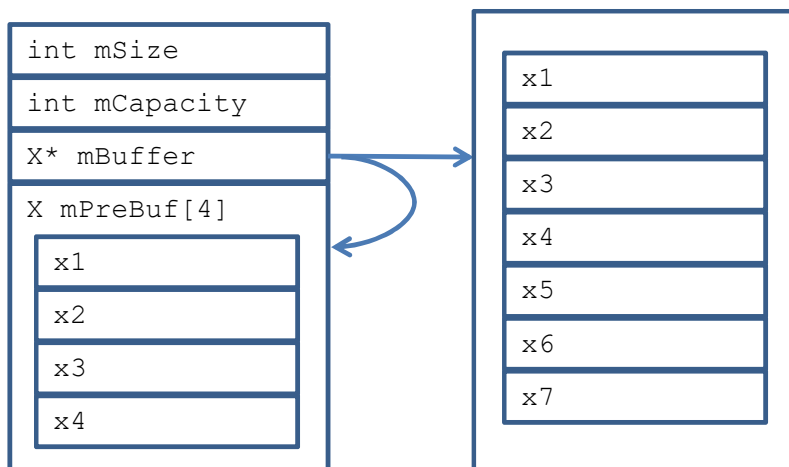
See Also

[CLIENT_Logout](#)

某摄像头设备的SDK (Software Development Kit) 开发手册

库函数接口 – 信息隐藏

自增数组的实现:



```
void XArray::push(X& x){
    if(mSize >= mCapacity)
        resize();
    mBuffer[mSize] = x;
    mSize++;
}
```

```
class XArray
{
private:
    int mSize;
    int mCapacity;
    X* mBuffer;
#define PRE_BUF_SIZE 4
    X mPreBuf[PRE_BUF_SIZE];

    void resize();
public:
    XArray() {
        mSize = 0;
        mCapacity = PRE_BUF_SIZE;
        mBuffer = mPrebuf;
    }
    int size() { return mSize; }
    void push(X& x);
    X* getHead() { return mBuffer; }
    ...
};
```

- 库函数的接口应该隐藏复杂的内部实现细节，暴露给调用者的概念尽可能的少。

库函数接口 – 资源管理

```
char* getFileLine(FILE* f){ // 获取文件中下一个非空，非注释的行
    char* buffer = new char[BUF_SIZE];
    char* p;
    while((p = fgets(buffer, BUF_SIZE, f)) != NULL){
        while(isspace(*p)) p++; // 找到第一个非空白字符
        if(*p == 0 || *p == '#') continue; // 忽略空行或者#开头的注释行
        return buffer;
    }
    return NULL;
}

...
char* p = getFileLine(f);
... // 处理p
delete p; // 需要删除p
```

```
char* getFileLine(FILE* f, char* buffer, int bufferSize){ // 获取文件中下一个非空，非注释的行
    char* p;
    while((p = fgets(buffer, bufferSize, f)) != NULL){
        while(isspace(*p)) p++; // 找到第一个非空白字符
        if(*p == 0 || *p == '#') continue; // 忽略空行或者#开头的注释行
        return buffer;
    }
    return NULL;
}

...
char buffer[1024];
char* p = getFileLine(f, buffer, sizeof(buffer));
... // 处理p, 不需要删除p
```

- 库函数不能偷偷申请资源，如果申请应该负责释放。应该让调用者自己管理资源。
- 明确这个谁申请谁释放的原则，避免接口对资源申请释放权的误解。

库函数接口 – 错误处理

```
int validateIpAddress(const char* ipAddress){
    ...
    if(发现了某个错误)
        exit(1)
    ...
}
```

```
int validateIpAddress(const char* ipAddress){
    ...
    if(发现了某个错误)
        printf("ip address %s 有某个错误\n", ipAddress)
    ...
}
```

```
int validateIpAddress(const char* ipAddress){
    ...
    if(发现了某个错误)
        MessageBox(hWin, "某某格式错误");
    ...
}
```

```
int validateIpAddress(const char* ipAddress){
    ...
    if(发现了某个错误)
        return IP_ERROR_XXX;
    if(发现了另一个错误)
        return IP_ERROR_YYY;
    ...
}
```

- 库函数接口应该安静地返回不同的返回值，让调用者根据返回值做不同的处理。
- 而不是在实现中越俎代庖自己控制程序行为，自己输出错误信息。

库函数接口 – 兼容性

ascToBcd函数的第一个版本的实现

```
// asc数字转bcd码, 例如"123456" => "\x21\x43\x65"  
// 如果是奇数位, 补0, 例如"12345" => "\x21\x43\x05"  
void ascToBcd(char* bcdResult, const char* ascDigits);
```

需求有变化, 奇数位的时候不一定补0, 可能会要求补F, 例如: "12345" => "\x21\x43\xF5"
所以对库函数ascToBcd做了修改, 增加了fillValue参数

```
// asc数字转bcd码, 例如"123456" => "\x21\x43\x65"  
// 如果是奇数位, 按照参数fillValue的取值进行补充  
// 例如fillValue为0, 则"12345" => "\x21\x43\x05"  
// 例如fillValue为0x0F, 则"12345" => "\x21\x43\xF5"  
int ascToBcd(char* bcdResult, const char* ascDigits, int fillValue);
```

新增的fillValue参数破坏了这个库函数的兼容性, 会导致原来使用这个接口的程序编译出错

正确的做法是: 为了库函数接口ascToBcd保持兼容, fillValue参数设置默认值0

```
// asc数字转bcd码, 例如"123456" => "\x21\x43\x65"  
// 如果是奇数位, 按照参数fillValue的取值进行补充  
// 例如fillValue为0, 则"12345" => "\x21\x43\x05"  
// 例如fillValue为0x0F, 则"12345" => "\x21\x43\xF5"  
// fillValue默认值为0  
int ascToBcd(char* bcdResult, const char* ascDigits, int fillValue = 0);
```

库函数接口 - 回调函数

应用程序

```
// 应用程序调用库函数setEventHandler  
// 设置回调函数入口  
...  
setEventHandler(OnEvent);  
...
```

```
// 回调函数，对事件进行处理  
void OnEvent(Event* event)  
{  
    // 应用程序编写的事件处理代码  
}
```

库

```
// 定义函数指针类型  
typedef void EventHandler(Event* event);  
  
// 定义一个全局变量保存回调函数入口  
EventHandler g_eventHandler;  
  
// 库函数setEventHandler的实现  
void setEventHandler(EventHandler* handler)  
{  
    // 将应用程序设置的回调函数入口保存在g_eventHandler变量  
    g_eventHandler = handler;  
}
```

```
...  
// 事件发生了，调用事件处理回调函数  
g_eventHandler(event);  
...
```

发生事件

库函数接口的缺点(1)

应用程序和库的代码在同一个进程内运行，共享相同的地址空间。如果程序运行出现的一些神秘的 **BUG**，往往难以定位责任。

因此，库函数接口适合的应用场合：

- 可靠的库
- 个人项目
- 小团队内项目
- 合作良好的团队间项目

库函数接口 **不适合** 的应用场合：

- 使用不熟悉的团队的不可靠的库



库函数接口的缺点(2)

应用程序和库的代码在同一个进程内运行，如果应用程序对实时性要求较高，而库函数内可能有较长时间的阻塞操作，会引起严重的性能问题。

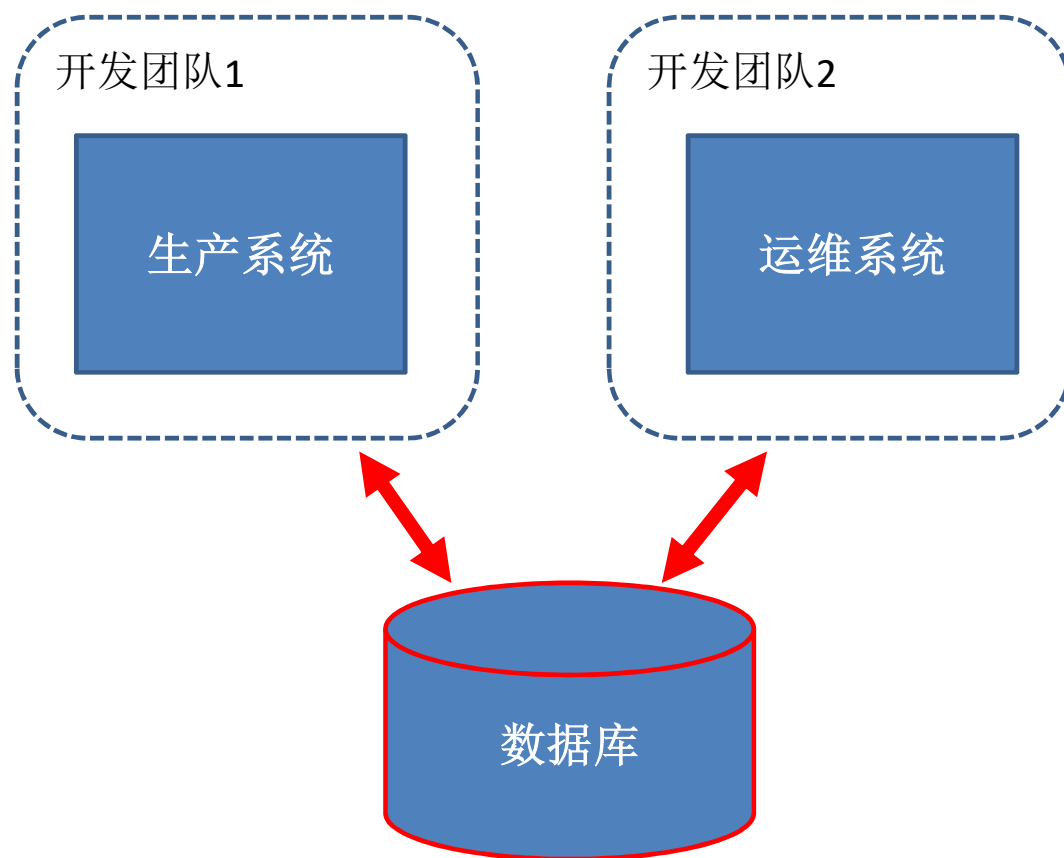
解决方案：

- 启动单独线程运行库函数
- 库函数提供异步接口

```
// 某设备开发SDK的登录接口，登录一个设备，登录成功才返回。  
LLONG CLIENT_Login(char *pchDVRIP, WORD wDVRPort  
    , char *pchUserName, char *pchPassword  
    , LPNET_DEVICEINFO lpDeviceInfo, int *error = 0  
);
```

```
// 某设备开发SDK的登录接口，调用之后立即返回。  
// 其中LPNET_DVR_USER_LOGIN_INFO含有回调函数指针，一旦登录成功调用回调函数。  
LONG NET_DVR_Login_V40(LPNET_DVR_USER_LOGIN_INFO pLoginInfo  
    , LPNET_DVR_DEVICEINFO_V40 lpDeviceInfo  
);
```

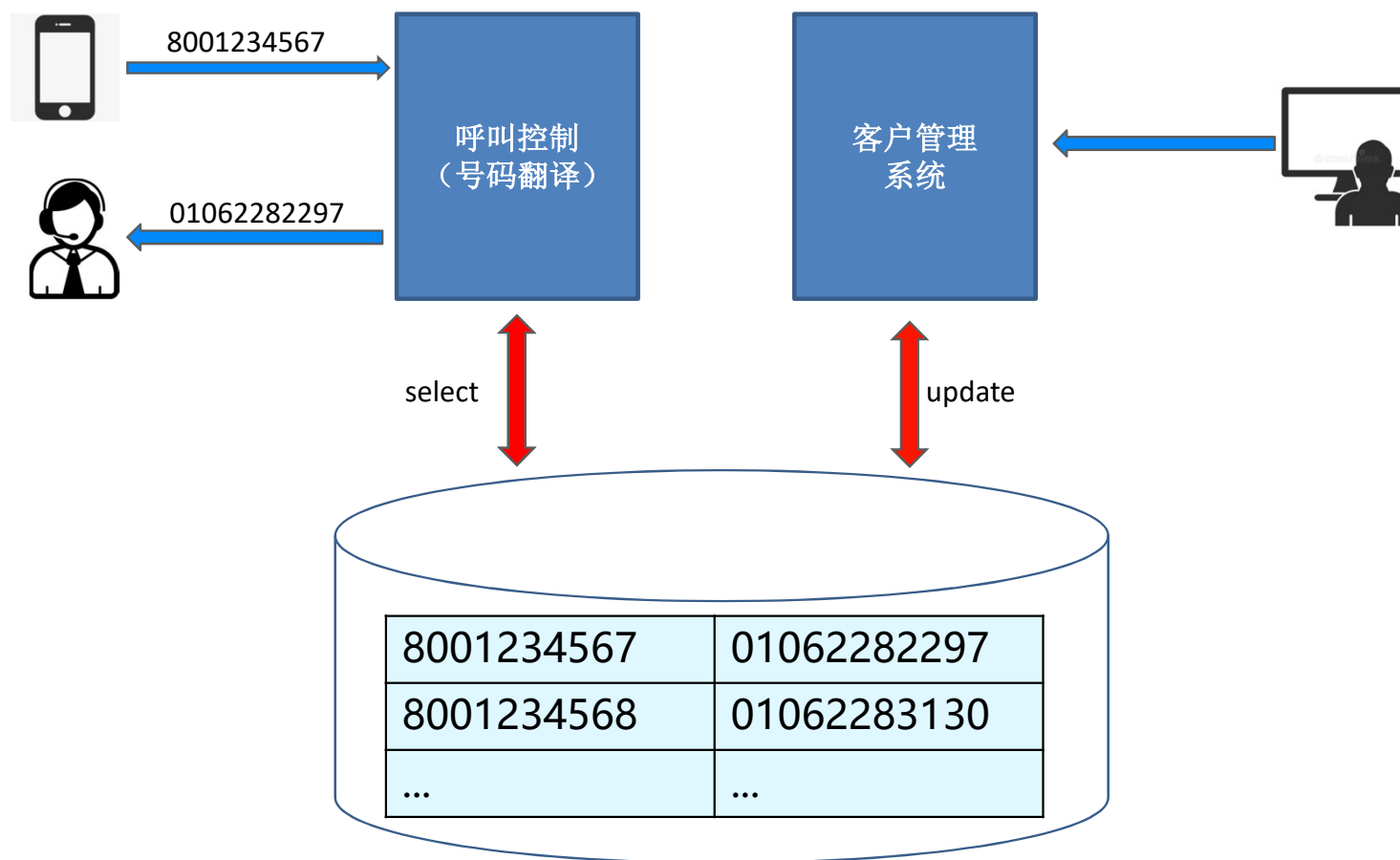

共享数据接口



接口说明（数据库表说明）：

- 每个表的名称，含义
- 每个字段的名称，类型，取值范围，含义
- 表和字段的关联逻辑

共享数据接口 – 示例



- 呼叫控制系统和客户管理系统共享了一个数据库表。这个数据库表的结构作为两个系统的接口。
- 客户管理系统将号码翻译对照表写入数据库。
- 呼叫控制系统根据用户打入的号码查询数据库表进行号码翻译，并呼出。

共享数据接口 – 存在缺点

思考一下：使用共享数据接口有哪些缺点？如何规避这些缺点导致的问题？

安全风险：

- 数据结构直接暴露给对方，信息安全无法保障。

性能不可控：

- 无法保证对数据的操作的方式和数量，性能无法保证。

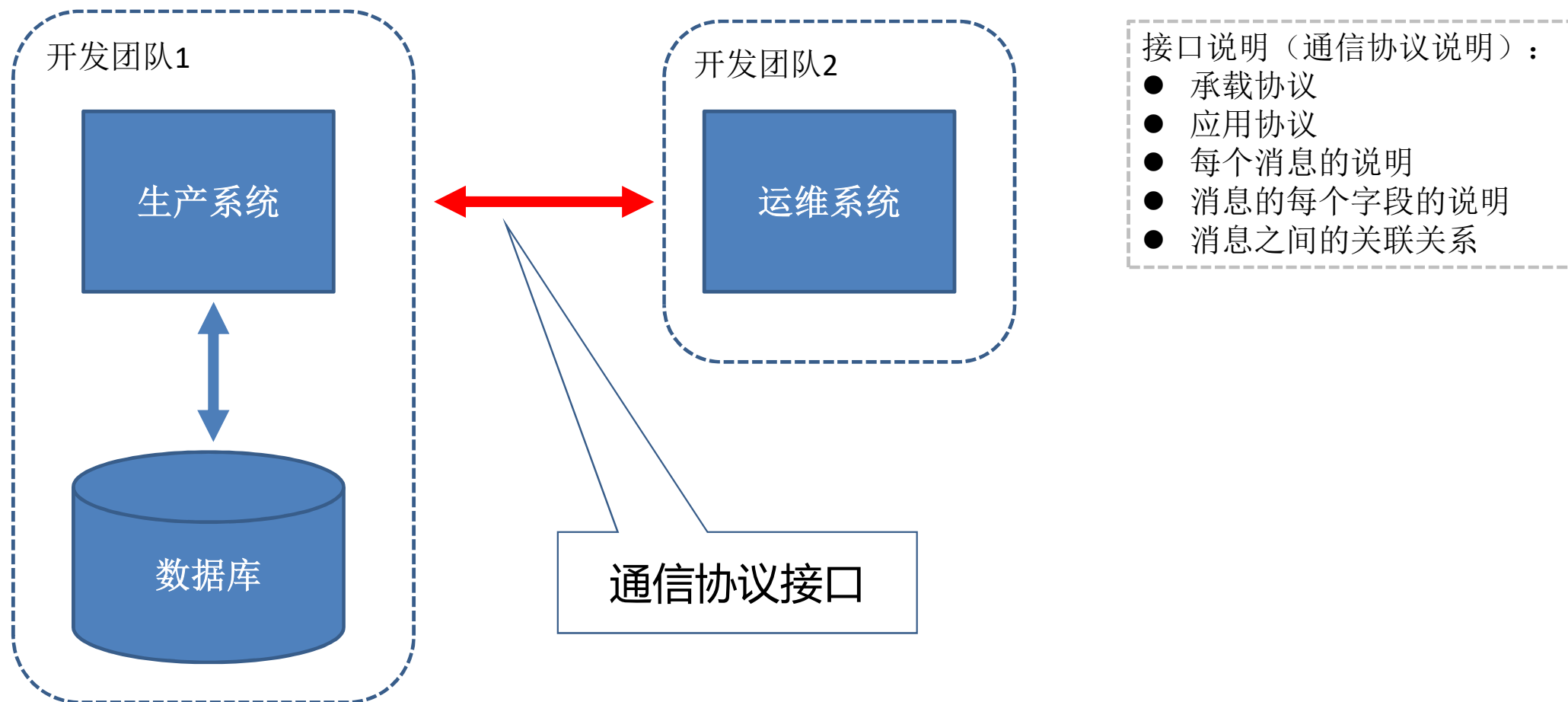
兼容性问题：

- 一方对数据格式的任何修改都会导致兼容性问题，使得系统的升级维护异常艰难。

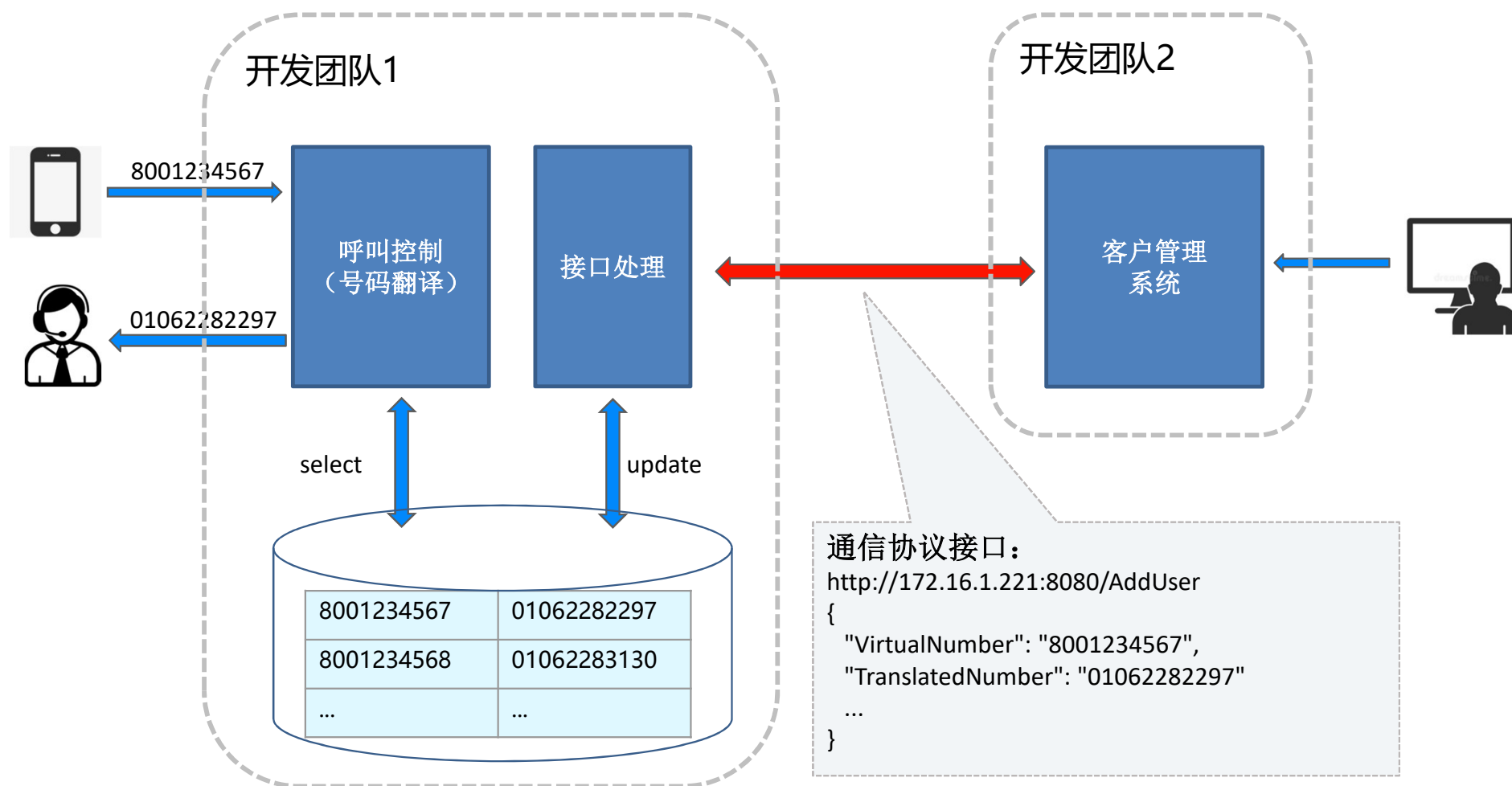
共享数据接口的适用范围：

- 共享数据接口仅适用于同一个人或者同一个小团队内部的模块之间。
- 一旦涉及不同团队（信息访问权限不同，沟通困难）之间，则不适合使用共享数据接口。
- 松耦合团队之间的接口，应该使用通信协议接口。

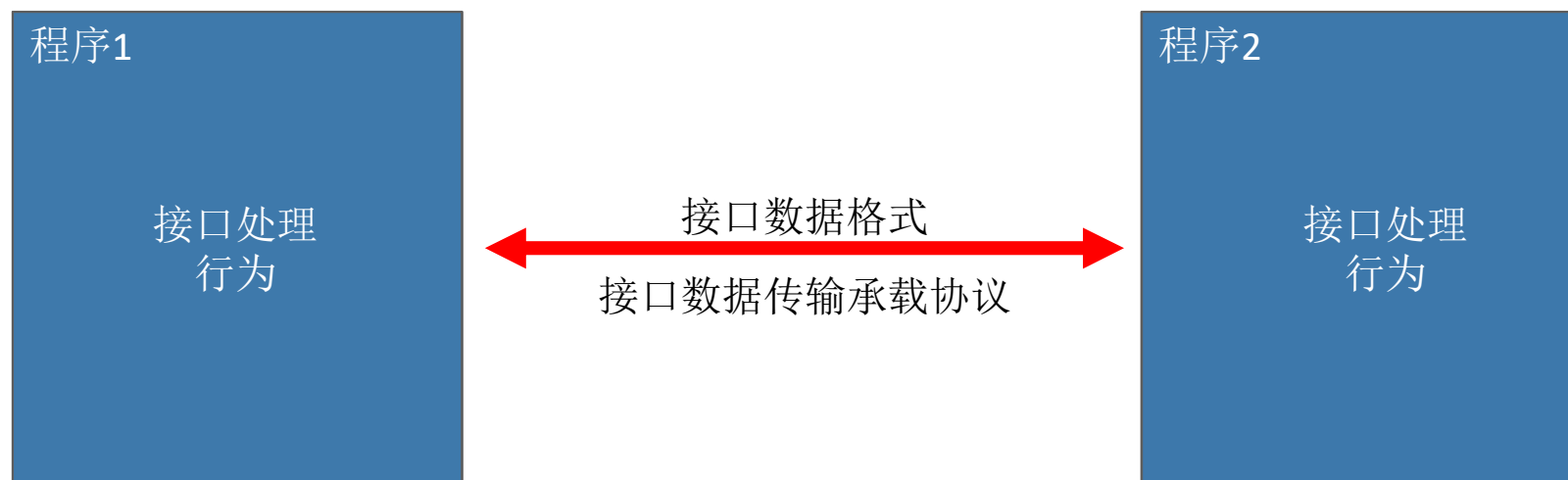
通信协议接口



通信协议接口



通信协议接口 – 如何定义



定义通信协议接口的三个要点

选择数据传输承载协议

定义接口数据格式

定义接口双方的处理行为

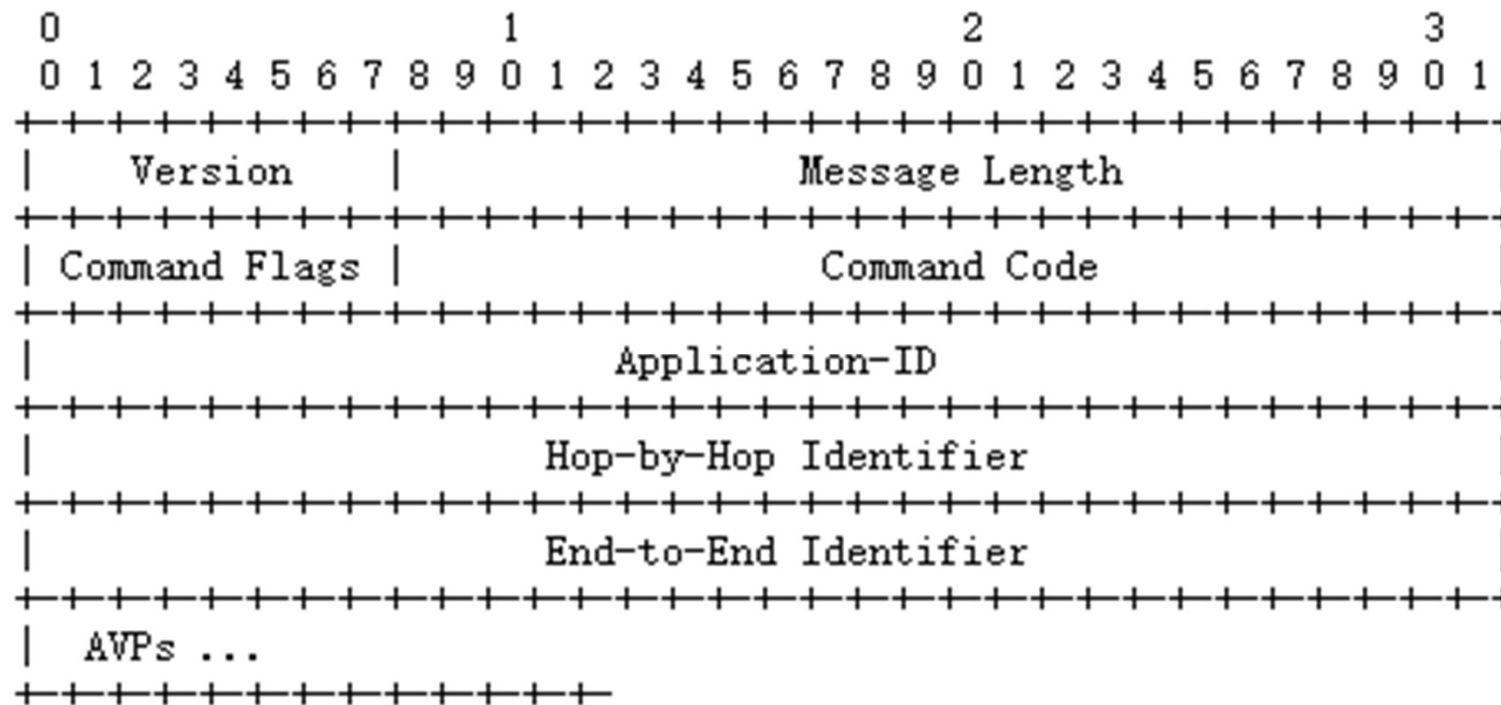
通信协议接口 – 主要承载协议

- 基于UDP, User Datagram Protocol
- 基于TCP, Transmission Control Protocol
- 基于更高层面的承载协议:
 - HTTP, 目前应用最广泛的一种无状态应答式接口承载协议。
 - WebSocket, 由HTTP衍生, 面向流数据。
 - RESTFUL, 基于HTTP, 一种特定的简约风格的接口设计。
 - MQTT, Message Queuing Telemetry Transport, 广泛应用于物联网领域的一种轻量级发布/订阅模式协议。
 - ZeroMQ, 一个基于消息队列的多线程网络库, 囊括了4中最通用的通信模型。
 - KAFKA, 基于消费者/生产者模式的实时消息中间件。
 - ...

通信协议接口 – 数据格式定义方法

- 直接描述二进制格式，对消息的每一个参数的格式和长度进行描述。
- 特殊文本格式，使用**BNF**范式进行描述。
- 基于某种通用格式，使用这种通用格式的特定描述方法进行描述：
 - ◆ **ASN.1: Abstract Syntax Notation 1**，抽象文法记法1（ITU-T）
 - ◆ **ProtoBuf: Protocol Buffer**（Google）
 - ◆ **XML: Extensible Markup Language**，可扩展标记语言
 - ◆ **JSON: JavaScript Object Notation**
 - ...

通信协议接口 – 数据格式定义范例



二进制格式的Diameter
协议
(RFC 6733)

通信协议接口 – 数据格式定义范例

```
INVITE sip:bob@biloxi.com SIP/2.0
Via: SIP/2.0/UDP pc33.atlanta.com;branch=z9hG4bKnashds8
To: Bob <bob@biloxi.com>
From: Alice <alice@atlanta.com>;tag=1928301774
Call-ID: a84b4c76e66710
CSeq: 314159 INVITE
Max-Forwards: 70
Date: Thu, 21 Feb 2002 13:02:03 GMT
Contact: <sip:alice@pc33.atlanta.com>
Content-Type: application/sdp
Content-Length: 147
```

```
v=0
o=UserA 2890844526 2890844526 IN IP4 here.com
s=Session SDP
c=IN IP4 pc33.atlanta.com
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

纯文本格式的SIP协议
(RFC 3261)

通信协议接口 – 数据格式定义范例

```
{  
    "format": "pcm",  
    "rate": 16000,  
    "dev_pid": 1537,  
    "channel": 1,  
    "token": "xxx",  
    "cuid": "baidu_workshop",  
    "len": 4096,  
    "speech": "xxx", // xxx为 base64 (FILE_CONTENT)  
}
```

基于json的百度语音识别接口

通信协议接口 – 数据格式定义范例

```
GetRequest-PDU ::=
  [0]
    IMPLICIT SEQUENCE {
      request-id
        RequestID,

      error-status          -- always 0
        ErrorStatus,

      error-index          -- always 0
        ErrorIndex,

      variable-bindings
        VarBindList
    }
```

基于ASN.1描述的SNMP协议
(RFC 1157)

通信协议接口 – 数据格式定义范例

```
message Lane_PB {  
    required int32 laneID = 1;  
    optional LaneAttributes_PB laneAttributes=2;  
    optional AllowedManeuvers_PB maneuvers=3;  
    optional ConnectsToList_PB connectsTo=4;  
    optional SpeedLimits_PB speedLimits=5;  
    optional PointList_PB points =6;  
}
```

基于ProtoBuf描述的一个地图接口

通信协议接口 – 数据格式定义范例

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap"
  xmlns:cwmp="urn:dslforum-org:cwmp-1-0">
  <soap:Body>
    <cwmp:Request>
      <argument>value</argument>
    </cwmp:Request>
  </soap:Body>
</soap:Envelope>
```

基于XML的宽带设备管理协议片段 (TR069)

通信协议接口 – 接口的兼容性

老版本接口定义:

```
message Lane_PB {  
    required int32 laneID = 1;  
    optional LaneAttributes_PB laneAttributes = 2;  
    optional AllowedManeuvers_PB maneuvers = 3;  
    optional ConnectsToList_PB connectsTo = 4;  
    optional SpeedLimits_PB speedLimits = 5;  
    optional PointList_PB points = 6;  
}
```

定义的不太好的新版本接口:

```
message Lane_PB {  
    required int32 laneID = 1;  
    optional int32 laneWidth = 2;  
    optional LaneAttributes_PB laneAttributes = 3;  
    optional AllowedManeuvers_PB maneuvers = 4;  
    optional ConnectsToList_PB connectsTo = 5;  
    optional SpeedLimits_PB speedLimits = 6;  
    optional PointList_PB points = 7;  
}
```

新增的参数的id和上一个版本的某参数的id冲突

如果考虑接口兼容性, 新版本应该这样定义:

```
message Lane_PB {  
    required int32 laneID = 1;  
    optional LaneAttributes_PB laneAttributes = 2;  
    optional AllowedManeuvers_PB maneuvers = 3;  
    optional ConnectsToList_PB connectsTo = 4;  
    optional SpeedLimits_PB speedLimits = 5;  
    optional PointList_PB points = 6;  
    optional int32 laneWidth = 7;  
}
```

通信协议接口

